

Name: Kamraan Faiyaz Mulani

Roll no: 23101A0028

INFT A - BE, DC Theory

Assignment No: 1

Synchronization in Distributed Computing Systems: A Case Study on Klarna's Distributed Payment Infrastructure

1. Introduction

Distributed systems rarely fail because a single machine runs out of resources. More often, the trouble starts the moment two or more processes reach for the same piece of shared state at almost the same instant. That's the exact problem the paper "Distributed Computing Systems Synchronization Modeling for Solving Machine Learning Tasks" by Azarnova and Polukhin sets out to tackle. It takes a well-known mutual exclusion algorithm, Suzuki-Kasami, and builds it into the language of queuing theory and Markov chains. So rather than just describing how a token gets passed around, the model can actually tell you how long a request is likely to sit and wait, how big a queue might grow, and how the whole system behaves as the request load shifts.

For this assignment, since my name starts with K, I picked Klarna the Swedish fintech company best known for its buy-now-pay-later service and online payments as the company to look at. Klarna works well for this kind of analysis because it isn't one big monolithic application sitting on a single server. It's made up of hundreds of independent microservices running across multiple data centers, and all of them have to agree, in real time, on things like whether a payment actually went through, whether a customer has already used up their credit limit, and whether an order was genuinely completed. That is precisely the kind of shared-resource coordination problem the paper is built to describe.

2. Klarna's Distributed Application Environment

Klarna processes millions of checkout transactions a day across many countries and currencies. To support that scale, the company has built its backend as a large collection of microservices, each responsible for a narrow slice of the business logic, such as credit scoring, fraud checks, order management, payment settlement, and merchant notifications. These services do not run on one server; they are distributed across clusters that may span several geographic regions so that customers in Europe, the US, and elsewhere all get low-latency responses.

The tricky part is that a single checkout is not handled by one service acting alone. When a customer clicks "pay," a chain of services has to be triggered in a coordinated way: the risk engine needs to check the customer's history, the ledger needs to record the transaction, the

Name: Kamraan Faiyaz Mulani

Roll no: 23101A0028

INFT A - BE, DC Theory

Assignment No: 1

credit limit needs to be updated, and a confirmation needs to be sent back to the merchant, all before the customer's browser times out. If two of these services update the same customer record at the same time, or if a retry mechanism fires twice because a response was delayed rather than lost, the result can be a double charge, a mismatched balance, or an order that shows as "pending" forever on one system and "completed" on another.

This is essentially the same environment the research paper describes in its introduction: parallel processes generating streams of requests that need access to a shared resource, whether that resource is a training sample in a machine learning pipeline or, in Klarna's case, a customer's account balance.

3. Synchronization Challenges Identified

- Concurrent access to shared account and ledger data: Several services fraud detection, order confirmation, refund processing can end up trying to read or write the same customer or merchant record within milliseconds of one another. None of them necessarily know the others are doing it at the same time.
- Distributed locking without a single point of failure: A single central lock manager isn't really an option for Klarna. Put everything through one node and you've just built yourself a bottleneck and worse, a single point of failure that the whole system depends on across every region.
- Unpredictable queueing under load: Peak periods like Black Friday cause synchronization requests to spike hard and fast. The problem is there's no straightforward way to predict how long any given request will sit in the queue once that happens it just depends on however much load hits at that moment.
- Timeout and exception handling: If a service holding a lock crashes, hangs, or just responds slowly, everything else waiting on that resource can't be left hanging indefinitely. There needs to be some kind of cutoff a maximum wait time after which the system backs out cleanly instead of grinding into a deadlock.
- Cross-region latency: Since nodes are spread across different geographic regions, the round-trip time to broadcast a message and get an acknowledgment back isn't fixed it varies. That inconsistency makes timestamp-based coordination approaches a lot messier to rely on.
- Cross-region latency: because nodes are geographically distributed, the time to broadcast a synchronization message and receive an acknowledgment is not constant, which complicates any timestamp-based coordination scheme.

4. Applying the Paper's Synchronization Concepts

4.1 Token-based coordination instead of timestamps

The paper contrasts timestamp-based algorithms such as Lamport and Ricart-Agrawala with the Suzuki-Kasami approach, which uses a single circulating token to grant exclusive access instead of relying on synchronized clocks across nodes. For Klarna, this distinction matters a great deal. Keeping clocks tightly synchronized across services running in different data centers and cloud regions is genuinely hard, and small clock drifts can cause timestamp-ordered algorithms to make incorrect ordering decisions. A token-passing scheme, where a service must hold a token before it can modify a specific customer's balance, sidesteps the clock-synchronization problem entirely. Only the service currently holding the token for that account is allowed to write to it, and the token, along with the request queue attached to it, moves on once the operation is finished.

4.2 Modeling requests as a queuing system

The paper's real contribution is treating the synchronization mechanism as a single-line queuing system with a Poisson arrival process for incoming requests and an exponentially distributed service time. That framing translates naturally to Klarna's checkout flow: each attempt to lock a customer's account for an update can be treated as an arriving "customer" in queuing-theory terms, arriving at some average rate that depends on overall transaction volume, and being served at a rate that depends on how long the update logic actually takes to run. Once Klarna has that queuing model in place, it can calculate, using the same differential equations the paper builds up from equations (1) through (12), the expected number of pending lock requests at any moment, and more importantly, the probability that a request will have to wait longer than an acceptable threshold, say 200 milliseconds, before a checkout is considered too slow.

4.3 Markov chain modeling of system states

Section 3 of the paper models the synchronization process as a birth-death Markov chain, where the system moves between states representing how many processes are currently in or waiting for the critical section. This maps directly onto Klarna's lock-manager behavior: state n represents an idle account record with no pending updates, state $n+1$ represents one service currently holding the lock, and each additional state represents another service queued up behind it. Using the transition probability matrix and the infinitesimal generator matrix

Name: Kamraan Faiyaz Mulani

Roll no: 23101A0028

INFT A - BE, DC Theory

Assignment No: 1

described in equations (16) to (22) of the paper, Klarna's platform engineers could estimate, for any given account, the steady-state probability of finding the record locked, and use that to decide things like how aggressively a retry policy should back off, or whether a particular account (say, a high-volume merchant) needs a dedicated lock partition rather than sharing infrastructure with ordinary customer accounts.

4.4 Bounded waiting and exception handling

The paper's mixed-type waiting system, where a maximum waiting period is defined for each process and an exception is raised once that period expires, is directly applicable to Klarna's timeout design. Rather than letting a stuck lock hang indefinitely and stall an entire checkout, Klarna's services could adopt the same principle: define a maximum tolerable wait time for a lock request, and if the token has not arrived within that window, roll back gracefully, release any partial state, and either retry with backoff or surface a clear failure to the customer instead of leaving the transaction in limbo.

5. A Practical Comparison, Following the Paper's Approach

The original paper benchmarks its Suzuki-Kasami-plus-queuing-theory model (SKQS) against Apache Spark's built-in BitTorrent-style broadcast synchronization and finds that SKQS reduces mutual locks essentially to zero while cutting synchronization time for a 100 MB data transfer from about 500 ms to 100 ms. Applying the same comparative logic to Klarna's environment, one can imagine a similar experiment: measure the current rate of lock conflicts and retries per thousand checkout transactions under Klarna's existing distributed-lock approach, then measure the same metrics after introducing a token-based scheme modeled with the queuing and Markov formulas from the paper. The table below sketches what such a comparison might reasonably look like, based on the kind of improvement the paper reports for its own test system.

Synchronization Approach	Conflicting Updates per 1000 Transactions	Average Lock Wait Time
Current distributed-lock / retry approach	Noticeable, rises sharply under peak load	Variable, harder to bound during spikes
Token-based (Suzuki-Kasami) + queuing model	Close to zero, since only the token holder can write	Predictable, calculable from queuing formulas

Note: figures are illustrative and framed on the pattern reported in the source paper's Table 1; they represent the kind of improvement the model predicts rather than measurements from Klarna's actual production systems.

6. Expected Benefits for Performance, Consistency, and Scalability

- Consistency: Because only one service gets to hold the token for a given account at any moment, you basically eliminate the scenario where two services step on each other while updating the same balance or order status. It's not that conflicts become rare they stop being possible in the first place.
- Predictable performance: The queuing model gives Klarna's engineers actual numbers to work with expected wait time, expected queue length instead of timeout values that were more or less guessed at. That means service-level targets can be based on math rather than gut feel or trial and error.
- Scalability under peak load: With the Markov chain model, the team can run through "what if" scenarios ahead of time what happens to the queues if transaction volume doubles during a big seasonal sale, for instance. That means extra lock-management capacity can be added in advance, rather than scrambling to fix things after checkouts already started failing.
- Fault tolerance: bounding the waiting period and raising a controlled exception when it is exceeded prevents one slow or crashed service from freezing an entire chain of dependent transactions.

7. Conclusion

The Azarnova and Polukhin paper was written with machine learning workloads in mind, but the underlying idea, that a token-passing mutual exclusion algorithm becomes far more useful once it is described in the language of queuing theory and Markov chains, is not tied to any one domain. Klarna's distributed payment infrastructure faces the same core problem the paper addresses: many independent processes competing for access to a small number of shared resources, with real consequences if two processes touch the same resource at once. By adopting a token-based synchronization scheme and modeling it as a queuing system with a Markov chain describing its state transitions, Klarna could move from a reactive, retry-heavy approach to one where waiting times, queue lengths, and failure rates are things the system can predict and plan for in advance, rather than problems discovered only after they cause an outage.

References

Azarnova, T. V. and Polukhin, P. V. 2021. Distributed computing systems synchronization modeling for solving machine learning tasks. Journal of Physics: Conference Series, 1902, 012050.

Name: Kamraan Faiyaz Mulani

Roll no: 23101A0028

INFT A - BE, DC Theory

Assignment No: 1

Klarna Bank AB. Corporate technology overview and engineering blog (publicly available company documentation), accessed for general background on Klarna's microservice-based payment platform